NOVABLE

12-Month Roadmap to an Emergent-Level AI App Builder

North Star: Transform Novable from a webpage generator into an AI software engineer that can take a natural-language app idea, plan it, build the frontend/backend/database, run it, test it, fix failures, and deploy it.

Positioning: Build the app. Launch it. Grow it. Novable combines an AI app generator with the marketing and growth capabilities already being developed.

1. The Target

Today	12-Month Target
Prompt → webpage	Prompt → complete application
Mostly frontend	Frontend + backend + database
User manually handles errors	Novable runs, tests, detects and repairs errors
Deploy a site	Deploy a functioning production app
Marketing tools alongside the builder	App Builder + Growth Engine

2. The Core Loop

Prompt → Understand → Plan → Architect → Generate → Execute → Test → Debug → Repair → Deploy

This loop is more important than the number of agents. The goal is a reliable software-engineering system, not a dashboard containing dozens of disconnected AI features.

3. 12-Month Roadmap

Months 1–2: Full-Stack Foundation

Focus: Move beyond webpage generation.

• Generate frontend + backend + database from one prompt.

• Support authentication, CRUD, file uploads, environment variables and API routes.

• Create a structured project representation so Novable understands files, components, routes, schemas and dependencies.

• Introduce project workspaces and persistent context.

Exit milestone: Milestone: Novable can generate a real CRUD SaaS app with login, database and dashboard.

Months 3–4: Autonomous Execution

Focus: Make Novable capable of actually running what it builds.

• Sandboxed code execution.

• Automatic dependency installation.

• Start local/dev servers automatically.

• Read terminal/build/runtime errors.

• Run browser-level checks.

• Give the coding agent the error context and let it patch the project.

Exit milestone: Milestone: Prompt → app → run → error → repair → successful run.

Months 5–6: Self-Testing and Reliability

Focus: Turn code generation into an engineering loop.

• Generate tests from requirements.

• Test API endpoints, database behavior and critical UI flows.

• Use browser automation for end-to-end testing.

• Create regression tests after fixes.

• Add checkpoints/versioning so failed changes can be reverted.

• Track generation success rate and repair rate.

Exit milestone: Milestone: Novable can repeatedly repair common failures without manual coding.

Months 7–8: Complex Applications

Focus: Move from simple CRUD apps toward serious products.

• Real-time features and chat.

• Payments and webhooks.

• Search, filtering and pagination.

• Role-based permissions.

• Admin dashboards.

• Third-party APIs and integrations.

• Background jobs, queues and notifications.

• More sophisticated database relationships.

Exit milestone: Milestone: Novable can build a Tinder-like or marketplace-style application with multiple interacting systems.

Months 9–10: Emergent/Lovable-Class Product Layer

Focus: Make the builder feel like a complete AI development environment.

• Natural-language iterative editing of an existing app.

• Understand the current codebase before modifying it.

• Plan large changes before execution.

• Show progress and explain what changed.

• Git/version history and rollback.

• Preview environments and production deployment.

• Improve UX so a non-programmer can build without understanding the generated code.

Exit milestone: Milestone: A new user can describe a reasonably complex app and reach a working deployment with minimal intervention.

Months 11–12: Scale, Benchmark, Launch

Focus: Prove the system works outside your own examples.

• Create a benchmark suite of 50–100 app-generation tasks.

• Test unseen applications, not only apps used during development.

• Measure build success, time-to-deploy, repair rate and human intervention.

• Fix the most common failure classes.

• Launch publicly and collect real user feedback.

• Add usage limits, billing and production monitoring.

• Use successful generations and failures to improve prompts, planning and evaluation.

Exit milestone: Milestone: Novable is a credible AI app-builder product, not a demo.

4. App Complexity Ladder

Level	Benchmark	Required capability
1	Landing page	Frontend generation
2	Todo / dashboard	CRUD + database
3	SaaS application	Auth + DB + APIs + deployment
4	Marketplace	Payments + roles + search + admin
5	Tinder-like app	Profiles + matching + real-time chat + notifications
6	Large e-commerce app	Catalog + search + cart + checkout + orders + inventory + seller/admin systems
7	General app builder	Reliable generation of unfamiliar complex applications

5. The Technical System to Build

Planner — Turns natural-language requirements into structured features, constraints and milestones.

Architect — Chooses project structure, stack, database schema, APIs and integration strategy.

Coding Agent — Creates and edits the actual repository.

Execution Sandbox — Runs commands, installs packages and starts the application safely.

Browser Agent — Interacts with the running app and verifies real user flows.

Test Agent — Creates and executes unit, API and end-to-end tests.

Debugger/Repair Agent — Reads failures, identifies root causes, edits code and retries.

Version Manager — Creates checkpoints, diffs, rollback points and safe iterations.

Deployment Layer — Builds and deploys applications to production.

Project Memory — Maintains requirements, architecture decisions, prior errors and user intent.

Evaluation System — Measures whether the generated application actually satisfies the request.

6. Keep the Existing Growth Engine

Do not throw away the marketing work already completed. Treat it as the second half of Novable.

Build Engine	Growth Engine
App generation	Local SEO
Backend + database	Customer re-engagement
Authentication	WhatsApp campaigns
Payments	UPI/payment workflows
Deployment	Analytics and conversion optimization

7. What Success Looks Like After One Year

✓ A user can describe an unfamiliar application in natural language.

✓ Novable decomposes the request into a structured implementation plan.

✓ Novable creates a complete full-stack codebase.

✓ The app actually runs in a sandbox.

✓ Novable automatically detects common build/runtime/UI failures.

✓ Novable can repair failures and retry.

✓ Critical user journeys are automatically tested.

✓ The project can be deployed with minimal human intervention.

✓ The user can continue modifying the live project using natural language.

✓ Novable can then help market and grow the application.

8. Metrics to Track

Metric	Why it matters
First-pass build success	Does Novable generate a working app without repair?
Repair success rate	Can it recover from its own mistakes?
Human intervention rate	How much manual engineering is required?
Time to working app	How quickly does a user reach a usable result?
End-to-end test pass rate	Does the generated product actually work?
Unseen-task success	Can it build apps it was not specifically tuned for?
User retention	Do people return because the product is genuinely useful?
Paid conversion	Will customers pay for the capability?

9. The One-Year Rule

Do not optimize for the number of features. Optimize for the number of unfamiliar applications Novable can successfully build from a prompt.

A beautiful interface with 30 agents is not the goal. A user typing an app idea and receiving a functioning, tested, deployable application is the goal.

Long-term vision: Novable becomes an AI software engineer that builds, deploys and grows applications.

This roadmap is an ambitious engineering plan, not a guarantee of a company valuation or revenue outcome. A multi-million-dollar company additionally requires product-market fit, distribution, pricing, reliability and customers.